Learning Suspicion: A Naive Bayes Classifier

~Umme Hani Chakkiwala
25B4510

1. Introduction

We all get those messages. The ones that say you've won a prize you never entered, or that your account has been suspended and you need to click a link immediately. Most of us can spot them instantly, but have you ever wondered how our phone does it?

There is no list of banned words sitting inside our messages app. Nobody programmed it to be suspicious of the word "free." The filter learned on its own, purely from examples, using probability as its only tool.

This report traces exactly how that works. I will start with a small puzzle - a question about a man named Steve that sounds obvious but turns out to be wrong for almost everyone. That puzzle leads naturally to Bayes' Theorem. And then I will show how this is applied in a spam ham classifier model.

2. A Puzzle About Steve

Read this description carefully:

> *"Steve is very shy and withdrawn, invariably helpful but with little interest in people or in the world of reality. A meek and tidy soul, he has a need for order and structure, and a passion for detail."*

Is Steve more likely to be a librarian or a farmer?

Take a moment. Most people say librarian immediately by hearing shy, tidy, detail-oriented, loves order. It sounds exactly right.

But it is almost certainly wrong. And the reason why reveals something fundamental about how probability actually works.

2.1 The Question Nobody Asks

When we read Steve's description, we ask: *does this match a librarian?* The answer is yes, strongly. So we conclude: librarian.

But there is a question we forget to ask first:

How many librarians and farmers actually exist?

In the United States, there are roughly 100,000 librarians and 2,000,000 farmers. That is 20 farmers for every 1 librarian. Before reading a single word about Steve, the base odds already say there is a 95% chance he is a farmer - just because farmers are so much more common.

Now bring in the description. Suppose 40% of librarians match Steve's personality. And suppose 10% of farmers do too - some farmers are quiet and detail-oriented. So Steve's description does match librarians better. But watch what happens when we combine both pieces of information:

Librarian match: 1 × 0.40 = 0.40

Farmer match: 20 × 0.10 = 2.00

Out of a total of 2.40, the farmer's share is 2.00 : which is **83%**.

Steve is most likely a farmer. The prior , how common each group is , completely changed the answer.

2.2 What Just Happened

We just did something very specific. We started with a prior belief that farmers are 20 times more common. We observed evidence : Steve's description matches librarians better. We combined them and got an updated belief that was different from both.This process of starting with what you know, observing evidence, and updating your belief has a name.

This is **Bayes' Theorem.**

$$P(\text{hypothesis} \mid \text{evidence}) = \frac{P(\text{evidence} \mid \text{hypothesis}) \times P(\text{hypothesis})}{P(\text{evidence})}$$

In plain words: your updated belief = how well the evidence fits × how likely the hypothesis was to begin with.
The insight that most people miss and that Steve's example makes impossible to ignore is that the prior is not optional. Forgetting it is not a minor error. It gives us the completely wrong answer.

3. The Spam Classifier

Now here is the interesting part. The question we just asked about Steve:

> *"Given this description, is he more likely to be a librarian or a farmer?"*

is exactly the question a spam classifier asks about every message:

> *"Given these words, is this message more likely to be spam or ham?"*

3.1 What the Classifier Does

(I have attached the complete code of this classifier model in a separate jupyter file named Code-Spam_classifier.ipynb, in the same repo, and here I have explained some important parts of the code. And also the dataset on which this model has been trained and tested is also given in a separate file named SMSSpamCollection_Dataset.)

The classifier is trained on 5,000 labelled SMS messages from the UCI Spam Collection dataset. It reads them, counts everything, and builds a simple picture of the world:

In [15]:

```
arr = []
for i in range(len(df)):
  x=df.loc[i,"text"]
  x=clean_and_tokenize(x)
  arr.append([df.loc[i,"label"],x])
print(len(arr))
for i in range(5):
    print(arr[i])
```

```
5572
['ham', ['go', 'until', 'jurong', 'point', 'crazy', 'available', 'only', 'in', 'bugis', 'n', 'great', 'world', 'la', 'e', 'buffet', 'cine', 'there', 'got', 'amore', 'wat']]
['ham', ['ok', 'lar', 'joking', 'wif', 'u', 'oni']]
['spam', ['free', 'entry', 'in', '2', 'a', 'wkly', 'comp', 'to', 'win', 'fa', 'cup', 'final', 'tkts', '21st', 'may', '2005', 'text', 'fa', 'to', '87121', 'to', 'receive', 'en
try', 'questionstd', 'txt', 'ratetcs', 'apply', '08452810075over18s']]
['ham', ['u', 'dun', 'say', 'so', 'early', 'hor', 'u', 'c', 'already', 'then', 'say']]
['ham', ['nah', 'i', 'dont', 'think', 'he', 'goes', 'to', 'usf', 'he', 'lives', 'around', 'here', 'though']]
```

We split the data into a training set and a test set. The training set is used to build the vocabulary and count the occurrences of each word in spam and ham messages. The test set is used to evaluate the performance of the classifier.

In [8]:

```
# Taking first 5000 data as training and remaining as testset
trainingSet = {}
count = {'ham': 0, 'spam': 0}
testSet = []

vlist = []
index = -1

for entry in arr:
    index += 1
    word_set = set(entry[1])
    if entry[0] == 'ham':
        class_count = Counter({'spam': 0, 'ham': 1})
    else:
        class_count = Counter({'spam': 1, 'ham': 0})
    if index <= 4999:
        count[entry[0]] += 1
        for word in word_set:
            if word not in vlist:
                vlist.append(word)
            if word not in trainingSet:
                trainingSet[word] = class_count.copy()
            else:
                trainingSet[word] += class_count
    else:
        testSet.append(entry)
```

This is where the model actually learns. The first 5000 messages go into training, the remaining 572 go into the test set. For each training message, the loop uses `set(entry[1])`,a set instead of a list to avoid counting the same word twice in one message. For every unique word, it records which class the message belonged to by adding to a Counter. After processing all 5000 messages, `trainingSet` is a complete dictionary where every word maps to how many spam and ham messages contained it. This is the raw data that the likelihood function will later use.

"free" → appeared in 300 spam, 20 ham messages
"win" → appeared in 250 spam, 5 ham messages
"ok" → appeared in 8 spam, 1200 ham messages
"hello" → appeared in 10 spam, 800 ham messages

From this it computes two things. **First, the prior : how common is spam overall**

```
def class_prior(cls):
  return count[cls]/5000
print(class_prior('spam'))
print(class_prior('ham'))
```

```
0.1346
0.8654
```

P(spam) = 673 / 5000 = 0.1346

P(ham) = 4327 / 5000 = 0.8654

Only 13% of messages are spam. So before reading a single word, the classifier already leans toward ham .

Second, the likelihood: how characteristic is each word of each class?

```
alpha = 0.00001

def word_likelihood(term, category):
    if term in trainingSet:
        return (trainingSet[term][category] + alpha) / (count[category] + 2 * alpha)
    else:
        return alpha / (count[category] + 2 * alpha)
```

This is P(word | class) in Bayes' Theorem - the likelihood. For a known word it divides how many messages of that class contained it by the total messages of that class. The small value alpha is Laplace smoothing ,it adds a tiny amount to every count so that no word ever gets a probability of exactly zero.

3.2 Classifying a New Message:

When a new message arrives, say **"win free cash"**, the classifier

computes a score for both spam and ham. It starts with the log of the prior and adds the log likelihood of each word:

```python
def posterior_score(doc_terms, category):
    score = math.log(class_prior(category))
    for term in doc_terms:
        score += math.log(word_likelihood(term, category))
    return score
```

We use logs to avoid numerical underflow, multiplying hundreds of tiny decimals causes the product to round to zero. Adding logs is mathematically equivalent and avoids this problem.

Watch how the scores evolve as we read the message word by word:

	Spam Score	Ham Score	Leader
Start (prior only):	-2.004	-0.144	Ham
After "win":	-2.994	-7.044	Spam
After "free":	-3.804	-12.344	Spam
After "cash":	-5.014	-18.564	Spam → FINAL ANSWER

The prior said ham. But the words were so strongly associated with spam that each one shifted the probability further and further until ham was not even close. The classifier returns spam.

This is Bayes' Theorem being applied three times in a row once per word. Each word is a new piece of evidence updating the belief.

4. Steve vs Spam

At this point we might notice something. In Steve's example the prior won: even strong evidence couldn't overcome the fact that farmers are 20x more common. In the spam example the evidence won : even

though 87% of messages are ham, three spammy words flipped the answer completely.

These are not two different theorems. They are the same theorem in two different situations.

Bayes' Theorem is a balance between prior and evidence. Sometimes the prior is heavy enough to dominate. Sometimes the evidence is strong enough to override it. The theorem tells you exactly how much weight to give each and the answer changes depending on the numbers.

Steve: Prior very strong (20x more farmers) → Prior wins → Intuition wrong

Spam: Evidence very strong (words 50x more common in spam) → Evidence wins → Intuition right

What they share is the same underlying question: *given everything I know, what is the most probable explanation?*

5. Results

Tested on 572 messages it had never seen, the classifier achieved:

Accuracy: 97.55% , 558 out of 572 correct.

```
correct = 0
conf_matrix = {"ham-ham": 0, "ham-spam": 0, "spam-spam": 0, "spam-ham": 0}
for actual, words in testSet:
    guess = classify(words)
    correct += actual == guess
    conf_matrix[f"{actual}-{guess}"] += 1

print("Test accuracy:", correct / len(testSet))
print("Confusion matrix:", conf_matrix)
```

```
Test accuracy: 0.9755244755244755
Confusion matrix: {'ham-ham': 487, 'ham-spam': 11, 'spam-spam': 71, 'spam-ham': 3}
```

Only 14 messages were misclassified. The classifier caught 71 out of 74 spam messages , missing just 3. And it flagged only 11 legitimate messages as spam out of 498.

6. Conclusion

We started with a puzzle that most people get wrong. Steve sounds like a librarian. But once you ask how many librarians and farmers exist in the world, the answer flips completely. The prior matters and Bayes' Theorem is the tool that makes sure you never forget it.

We then showed that the same reasoning drives a real spam classifier. The prior tells the model that most messages are legitimate. The likelihood of each word updates that belief. The posterior makes the final call.

Probability does not just describe uncertainty. Used correctly, it resolves it.

THANK YOU!